🟢 SESIÓN 5 DTOs de salida + Contrato de API

"La base de datos NO es el contrato de la API."

Qué ya saben

- Entity (Book) + JPA + MySQL
- CRUD
- ResponseEntity (status codes)
- DTO de entrada BookRequestDTO + @Valid
- Handler de errores de validación

Qué falta naturalmente

👉 La API ya valida... pero sigue devolviendo **entities**.

Problema real:

- Si mañana cambia la entity, el JSON cambia → rompes clientes.
- Puedes exponer campos internos sin querer.

💡 Mensaje clave:

"La BD no es el contrato de la API."

Estructura del proyecto (Sesión 5)

```
com.cladsb1.books
 ├── controller
 │   └── BookController
 ├── entity
 │   └── Book
 ├── repository
 │   └── BookRepository
 ├── dto
 │   ├── request
 │   │   └── BookRequestDTO
 │   └── response
 │   └── BookResponseDTO
 ├── error
 │   ├── ApiError
 │   └── GlobalExceptionHandler
 └── BooksApplication
```

🟦 Bloque 1 — Teoría esencial: ¿por qué DTO de salida?

Entity

- representa cómo guardo datos
- puede tener campos técnicos
- puede cambiar por motivos internos

DTO response

- representa lo que la API promete
- es estable
- es "contrato" con el cliente

✅ Objetivo: devolver BookResponseDTO en vez de Book.

🟦 Bloque 2 — Crear DTO de salida

📁 dto/response/BookResponseDTO.java

```
package com.cladsb1.books.dto.response;

/**
 * DTO de salida: define exactamente qué campos ve el cliente.
 */
public record BookResponseDTO(
 Long id,
 String title,
 String author,
 String category
) {}
```

🟦 Bloque 3 — Mapping: Entity → ResponseDTO (concepto)

Hay dos estilos:

✅ Moderno: stream().map(...)

Utiliza programación funcional y method references para transformar colecciones de forma elegante.

✅ Clásico: for + lista

Usa bucles tradicionales y listas para realizar las transformaciones de manera explícita.

Esta sesión enseña el mapping "a mano" para entenderlo. (MapStruct lo veremos después.)

🟦 Bloque 4 — Controller

✅(VERSIÓN A) CON streams/lambdas

📁 controller/BookController.java (CON streams/lambdas)

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.repository.BookRepository;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookRepository bookRepository;

    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    /**
     * GET all -> devolvemos DTOs (no entities)
     */
    @GetMapping
    public ResponseEntity<List<BookResponseDTO>> getAll() {

        List<BookResponseDTO> result = bookRepository.findAll()
                .stream()
                .map(this::toResponseDTO) // method reference
                .toList();

        return ResponseEntity.ok(result);
    }


    /**
     * GET by id -> 200 con DTO o 404
     */
    @GetMapping("/{id}")
    public ResponseEntity<BookResponseDTO> getById(@PathVariable Long id) {

        return bookRepository.findById(id)
                .map(this::toResponseDTO)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    /**
     * POST -> recibimos DTO request validado, devolvemos DTO response
     */
    @PostMapping
    public ResponseEntity<BookResponseDTO> create(@Valid @RequestBody BookRequestDTO dto) {

        Book book = new Book(dto.title(), dto.author(), dto.category());
        Book saved = bookRepository.save(book);

        URI location = URI.create("/api/books/" + saved.getId());
        return ResponseEntity.created(location).body(toResponseDTO(saved));
    }

    /**
     * PUT -> actualiza si existe (200), si no 404
     */
    @PutMapping("/{id}")
    public ResponseEntity<BookResponseDTO> update(@PathVariable Long id,
                                                  @Valid @RequestBody BookRequestDTO dto) {

        return bookRepository.findById(id)
                .map(existing -> {
                    existing.setTitle(dto.title());
                    existing.setAuthor(dto.author());
                    existing.setCategory(dto.category());
                    Book saved = bookRepository.save(existing);
                    return ResponseEntity.ok(toResponseDTO(saved));
                })
                .orElse(ResponseEntity.notFound().build());
    }

    /**
     * DELETE -> 204 o 404
     */
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        if (!bookRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        bookRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    /**
     * Mapping centralizado (manual).
     * En sesión 9 lo sustituiremos por MapStruct.
     */
    private BookResponseDTO toResponseDTO(Book book) {
        return new BookResponseDTO(
                book.getId(),
                book.getTitle(),
                book.getAuthor(),
                book.getCategory()
        );
    }
}
```

✅ Ventaja: introducción "method reference" this::toResponseDTO .

✅ Controller (VERSIÓN B) SIN streams/lambdas

📁 controller/BookController.java (SIN streams/lambdas)

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.repository.BookRepository;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookRepository bookRepository;

    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @GetMapping
    public ResponseEntity<List<BookResponseDTO>> getAll() {

        List<Book> books = bookRepository.findAll();
        List<BookResponseDTO> result = new ArrayList<>();

        for (Book b : books) {
            result.add(toResponseDTO(b));
        }

        return ResponseEntity.ok(result);
    }

    @GetMapping("/{id}")
    public ResponseEntity<BookResponseDTO> getById(@PathVariable Long id) {

        Optional<Book> opt = bookRepository.findById(id);
        if (opt.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        BookResponseDTO dto = toResponseDTO(opt.get());
        return ResponseEntity.ok(dto);
    }

    @PostMapping
    public ResponseEntity<BookResponseDTO> create(@Valid @RequestBody BookRequestDTO dto) {

        Book book = new Book(dto.title(), dto.author(), dto.category());
        Book saved = bookRepository.save(book);

        URI location = URI.create("/api/books/" + saved.getId());
        return ResponseEntity.created(location).body(toResponseDTO(saved));
    }

    @PutMapping("/{id}")
    public ResponseEntity<BookResponseDTO> update(@PathVariable Long id,
                                                  @Valid @RequestBody BookRequestDTO dto) {

        Optional<Book> opt = bookRepository.findById(id);
        if (opt.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        Book existing = opt.get();
        existing.setTitle(dto.title());
        existing.setAuthor(dto.author());
        existing.setCategory(dto.category());

        Book saved = bookRepository.save(existing);
        return ResponseEntity.ok(toResponseDTO(saved));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {

        boolean exists = bookRepository.existsById(id);
        if (!exists) {
            return ResponseEntity.notFound().build();
        }

        bookRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    private BookResponseDTO toResponseDTO(Book book) {
        return new BookResponseDTO(
                book.getId(),
                book.getTitle(),
                book.getAuthor(),
                book.getCategory()
        );
    }
}
```

🟦 Bloque 5 — Tabla: operación → status

Operación	Endpoint	Status	Body
GET all	GET /api/books	200	List<BookResponseDTO>
GET by id existe	GET /api/books/{id}	200	BookResponseDTO
GET by id no existe	GET /api/books/{id}	404	—
POST válido	POST /api/books	201	BookResponseDTO
POST inválido	POST /api/books	400	ApiError (handler)
PUT existe válido	PUT /api/books/{id}	200	BookResponseDTO
PUT no existe	PUT /api/books/{id}	404	—
PUT inválido	PUT /api/books/{id}	400	ApiError
DELETE existe	DELETE /api/books/{id}	204	—
DELETE no existe	DELETE /api/books/{id}	404	—

🟦 Bloque 6 — Errores típicos

❌ Error 1: seguir devolviendo entity "por comodidad"

```
public ResponseEntity<Book> getAll() { ... }
```

👉 Estás exponiendo tu BD.

❌ Error 2: mapping repetido en 5 sitios

Solución docente:

- centralizar mapping en un método toResponseDTO
- (MapStruct en una sesión posterior)

❌ Error 3: confundir DTO request con response

- Request: lo que entra (validación)
- Response: lo que sale (contrato)

🟦 Bloque 7 — Cierre conceptual

💡 Mensaje final:

"La base de datos NO es el contrato de la API."